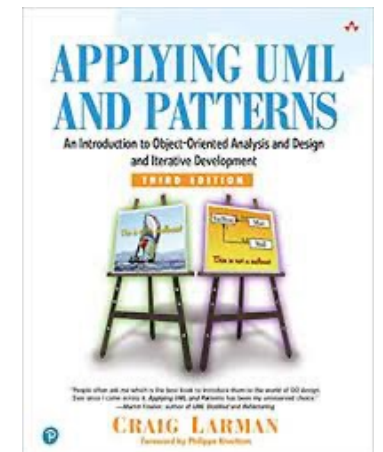


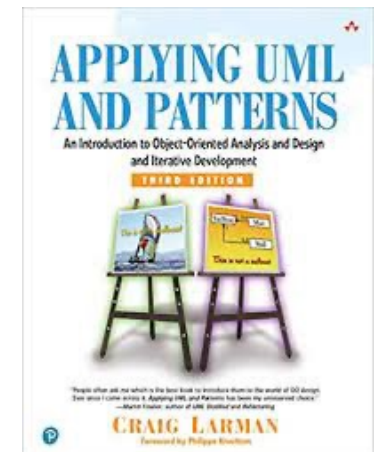
Resumo do livro Utilizando UML e Padrões

Fonte: Capítulos 1, 3, 6 e 9 do livro Utilizando UML e Padrões.
Este resumo foi utilizado na aula 12 do curso Orientação a Objetos, 2023.

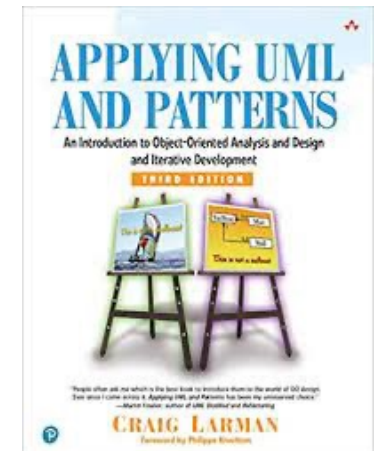


Parte 1 - Introdução

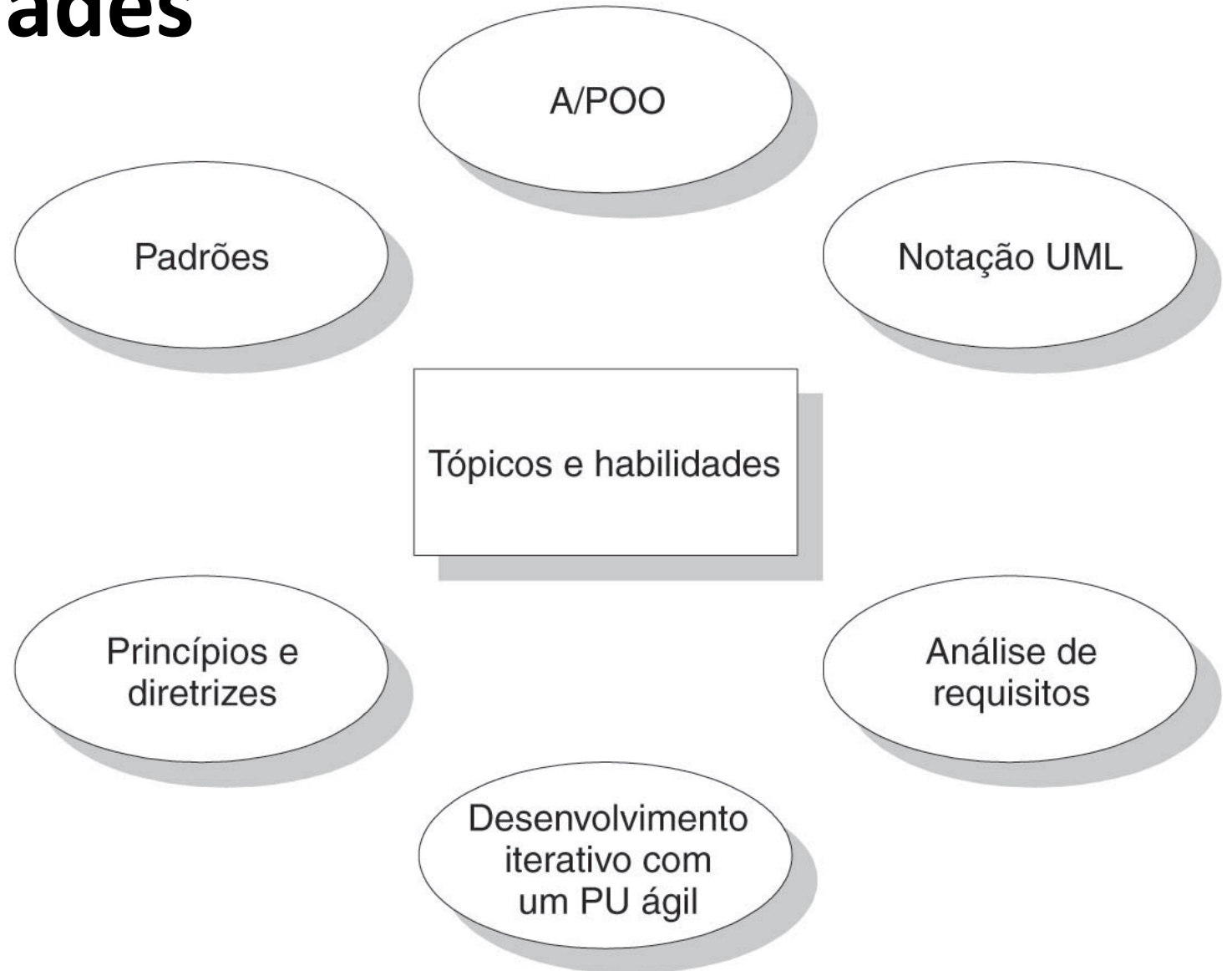
Fonte: Capítulos 1, 3, 6 e 9 do livro Utilizando UML e Padrões.
Este resumo foi utilizado na aula 12 do curso Orientação a Objetos, 2023.



Capítulo 1: Análise e design orientado a objetos



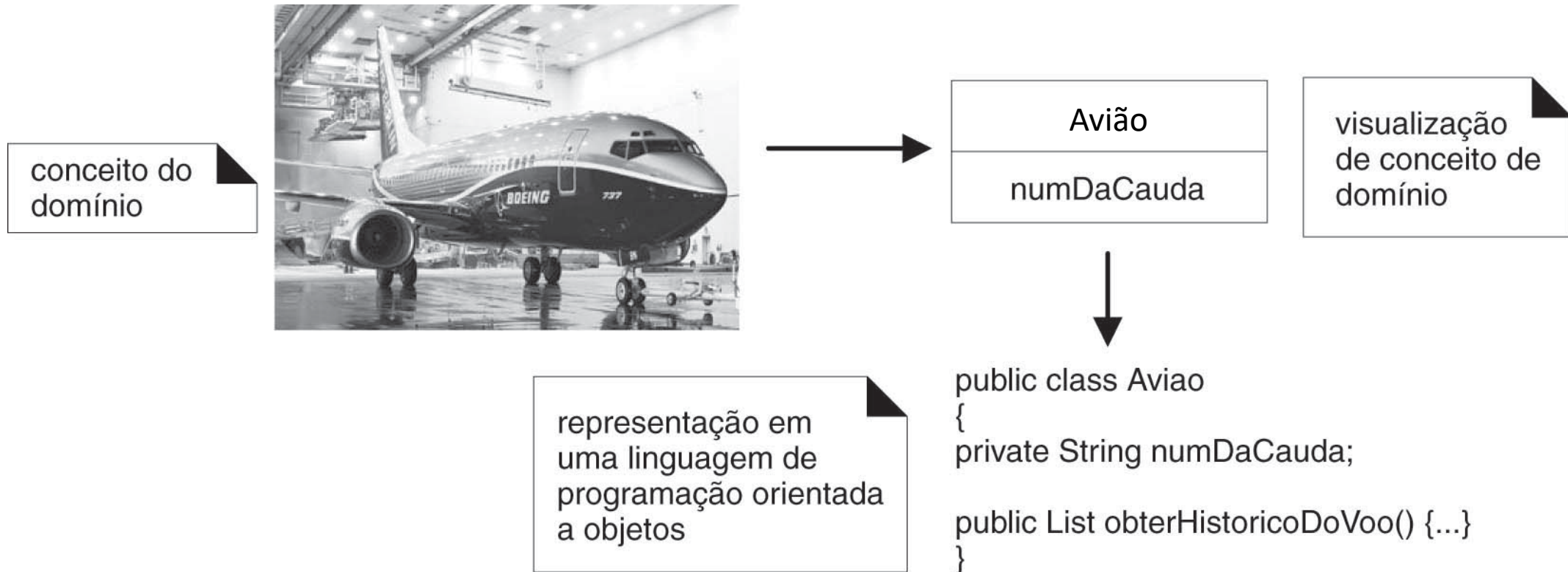
Tópicos e habilidades



O objetivo mais importante do aprendizado

“Uma habilidade crucial no desenvolvimento OO é atribuir, adequadamente, responsabilidades aos objetos de software”.

OO enfatiza representação de objetos



A tail number (número da cauda) is an **alphanumeric code** between **two** and **six characters** in **length** used to **identify** a specific airplane.



Processo de exemplo para análise e *design* OO

Definir casos
de uso

Definir modelo
de domínio

Definir diagramas
de interação

Definir diagramas
de classes de
projeto

Modelo de domínio parcial (jogo de dados)

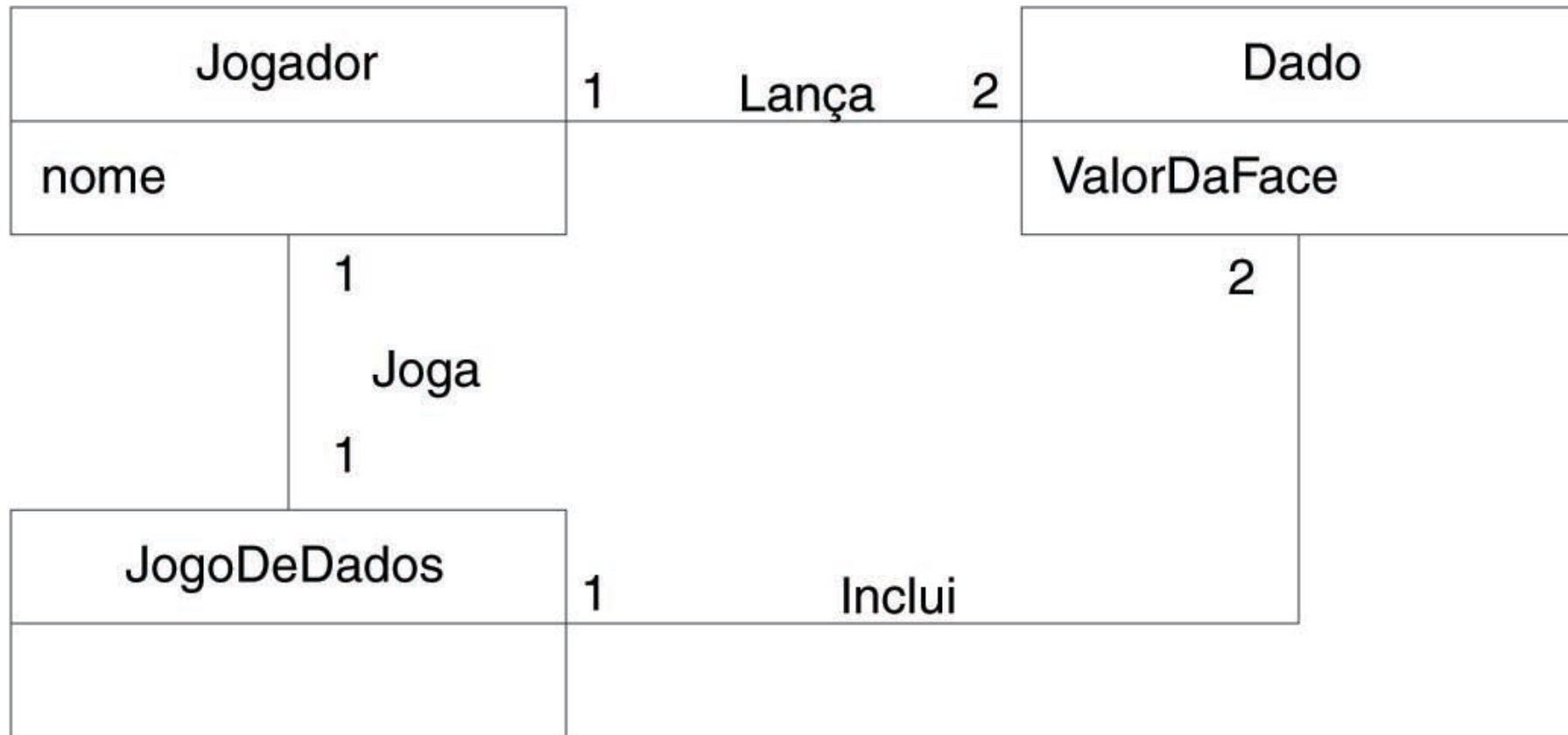


Diagrama de sequência UML

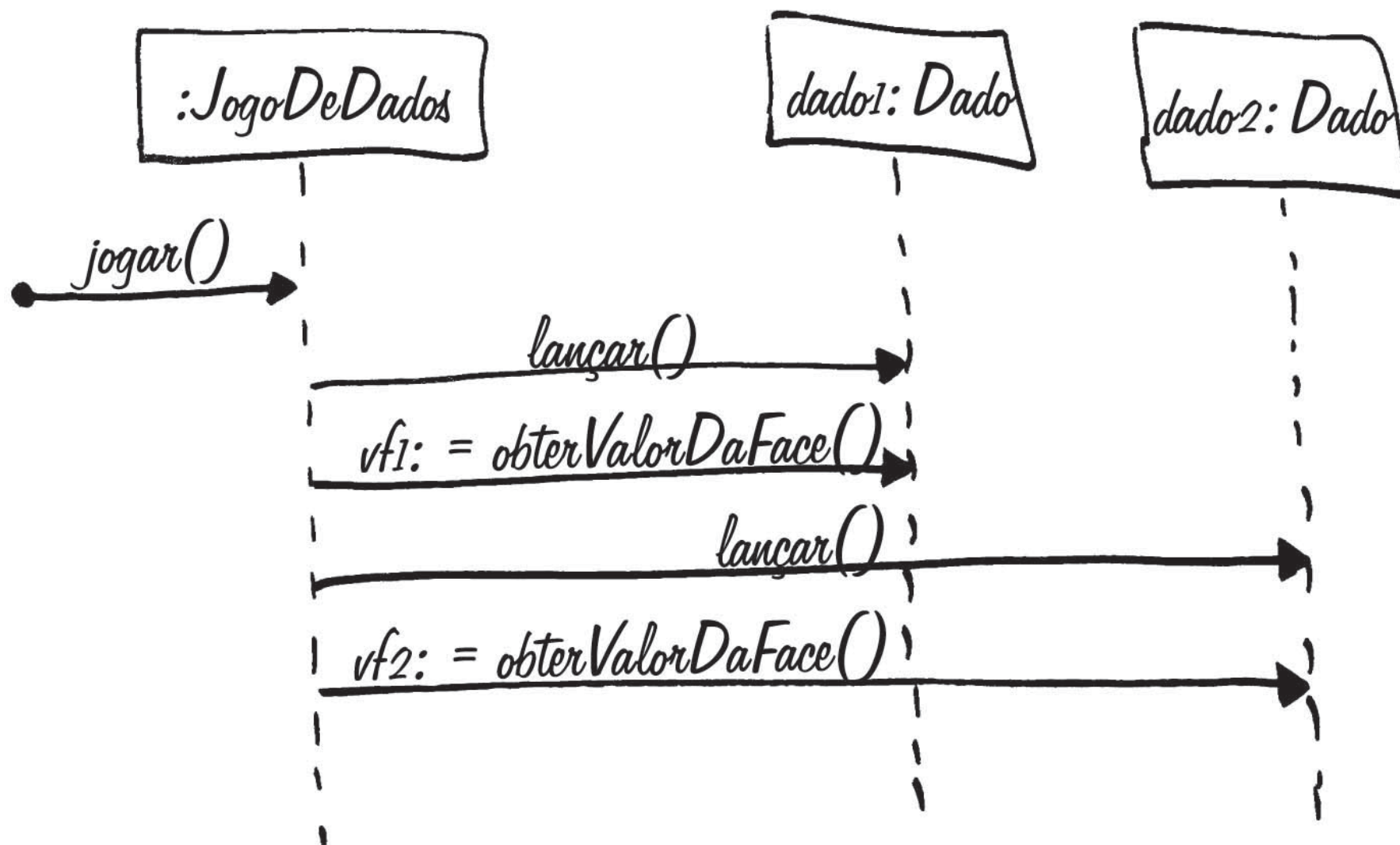
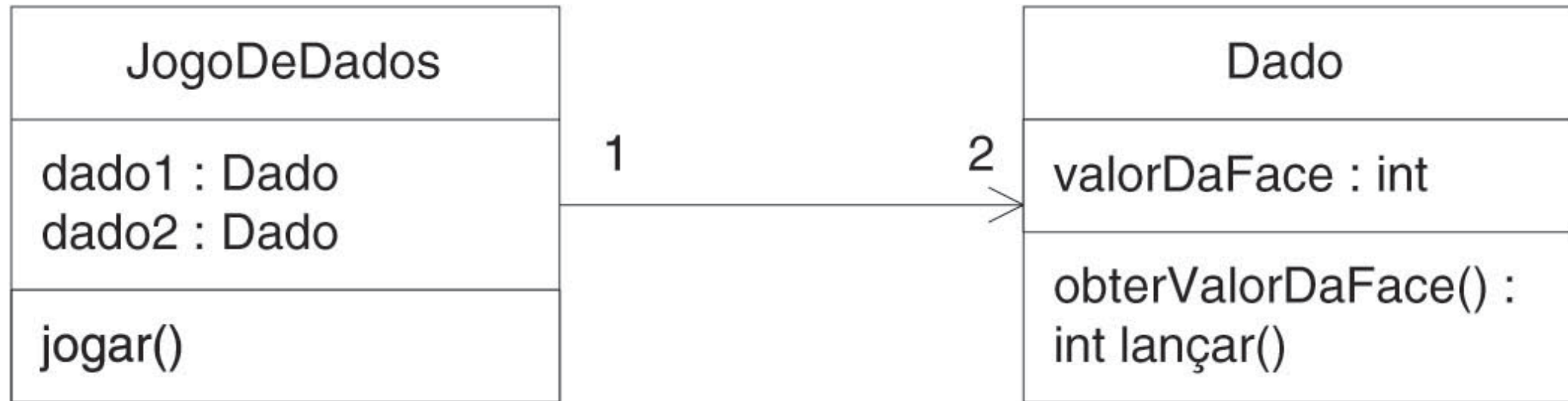


Diagrama de classe parcial em nível de *design*

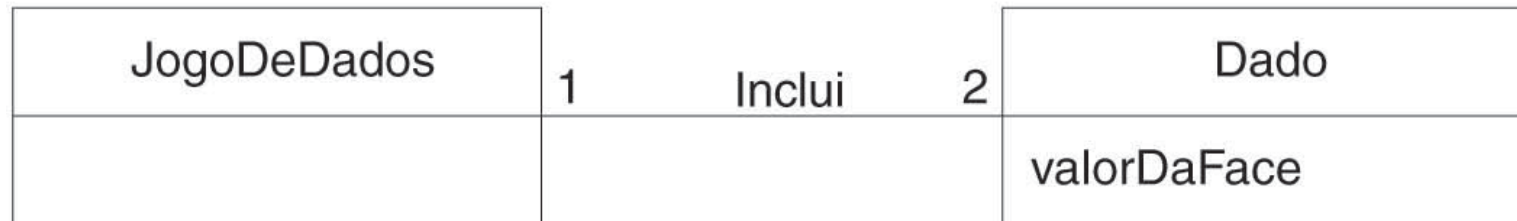


O que é UML?

*“A Linguagem de Modelagem Unificada (UML) é uma **linguagem visual** para especificar, construir e documentar os artefatos dos sistemas”.*

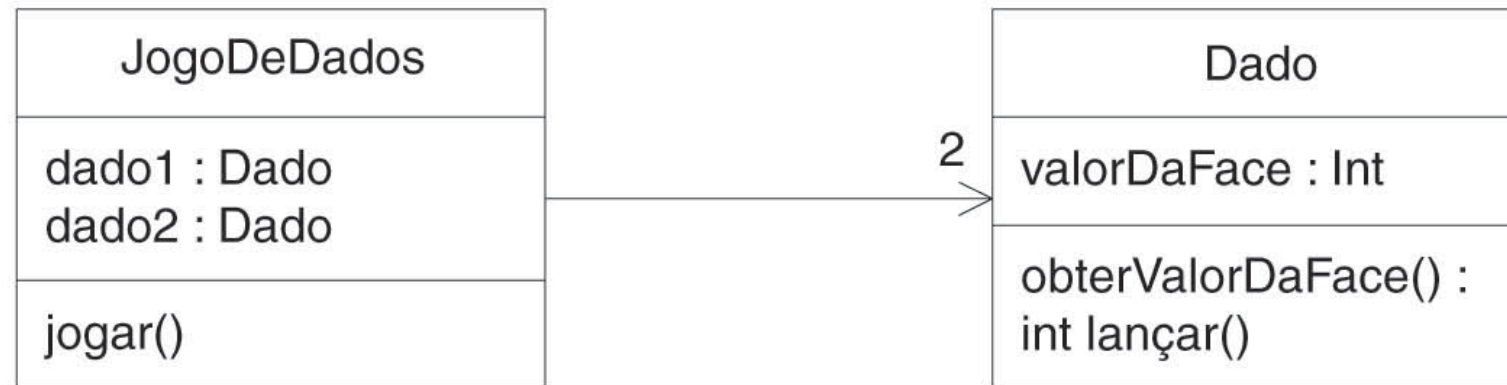
[OMG03]

Perspectivas distintas com UML



Perspectiva conceitual
(modelo do domínio)

Notação do esboço do
Diagrama de Classe UML
usada para visualizar os
conceitos do mundo real



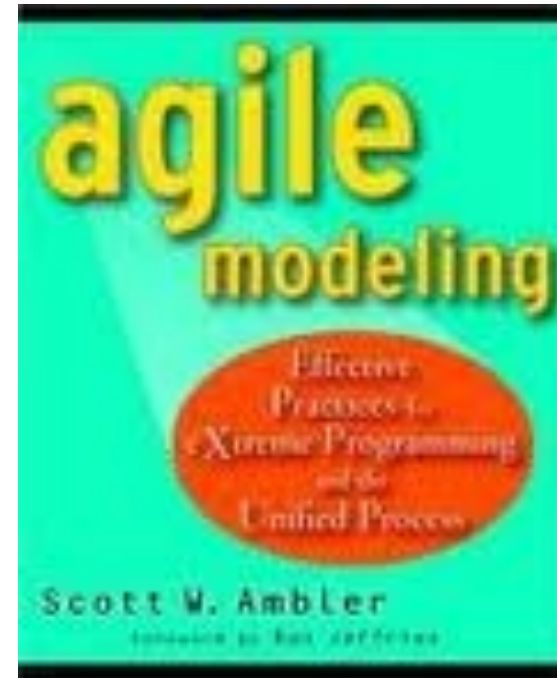
Perspectiva de
especificação ou
implementação (diagrama
de classe de projeto)
Notação do Diagrama de
Classe UML bruto usado
para visualizar os elementos
de software

O que é modelagem ágil?

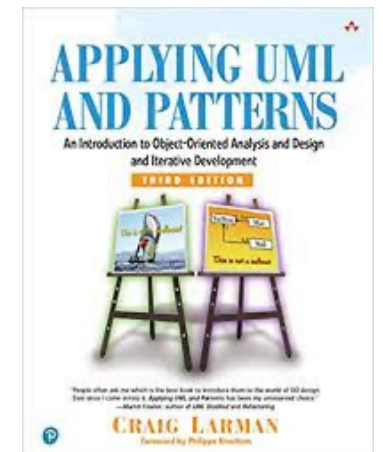
“A finalidade da modelagem é principalmente **entender** e não **documentar**”.

O que é modelagem ágil?

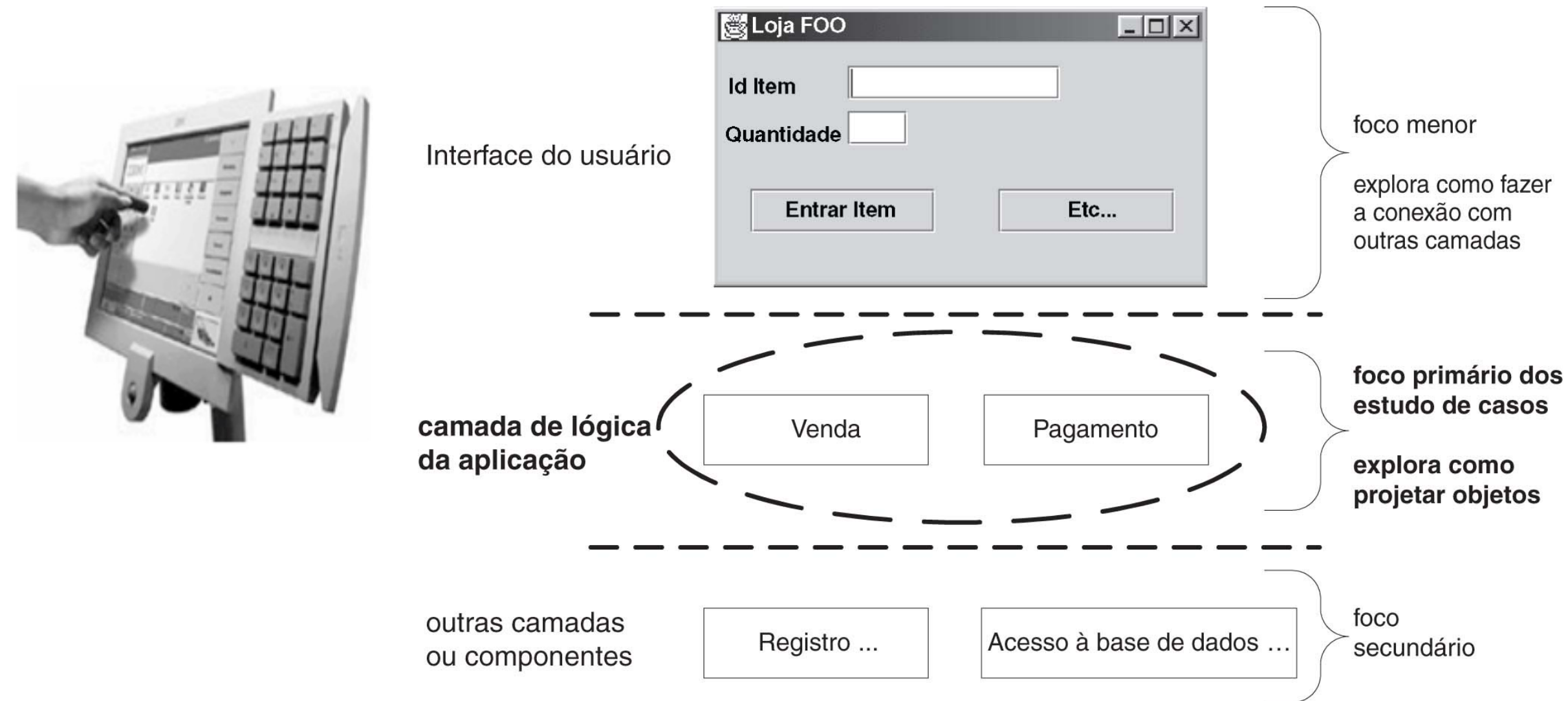
- Esta visão da finalidade da modelagem é conhecida como **modelagem ágil**.



Capítulo 3: Estudos de caso



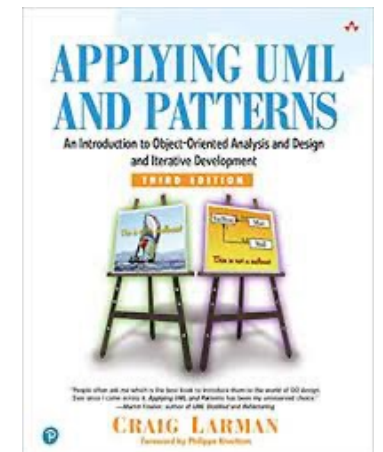
Camadas e objetos em um sistema OO



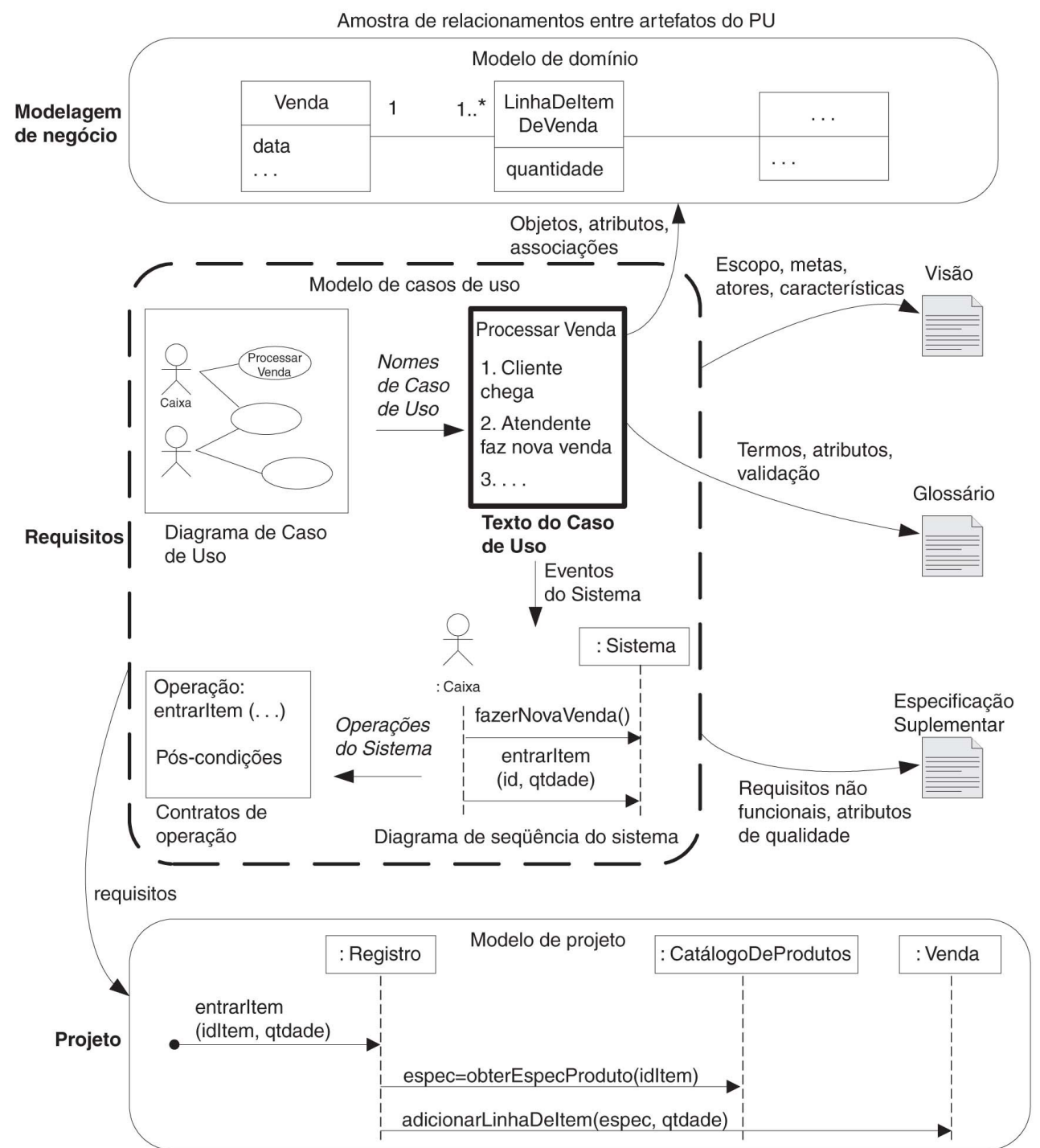
Simulador *monopoly* (Banco imobiliário)



Capítulo 6: Casos de uso



Exemplo de disciplinas e artefatos no contexto do Process Unificado



Definição: casos de uso

*“Um **caso de uso** é uma **coleção de cenários** relacionados de **sucesso** e **insucesso**, que descrevem um **ator** usando um **sistema** como meio para atingir um **objetivo**”.*

Cenário de sucesso e insucesso

Cenário de sucesso principal (ou Fluxo básico)	1. Cliente chega à saída do PDV com seus bens e serviços a adquirir. 2. Caixa começa uma nova venda. 3. Caixa insere o identificador do item. 4. Sistema registra a linha de item de venda e apresenta uma descrição do item, seu preço e total parcial da venda. Preço calculado segundo um conjunto de regras de preços. <i>Caixa repete os passos 3 e 4 até que indique ter terminado.</i>
--	--

3a. ID do item inválido (não encontrado no sistema):

1. Sistema avisa o erro e rejeita a entrada.
2. Caixa responde ao erro

2a. Existe um ID do item legível a uma pessoa humana (por exemplo, CUP numérico)

Diagrama parcial de casos de uso

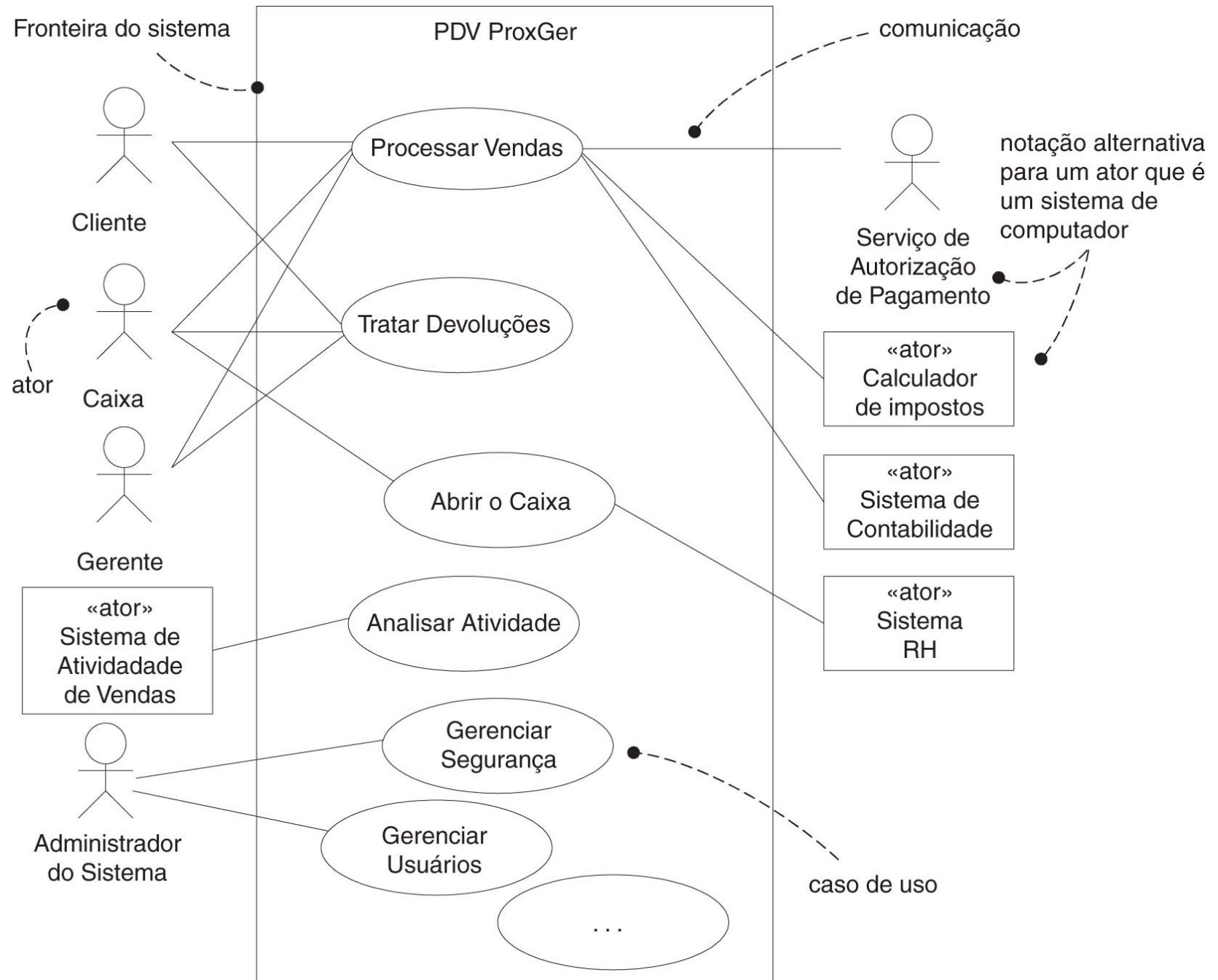
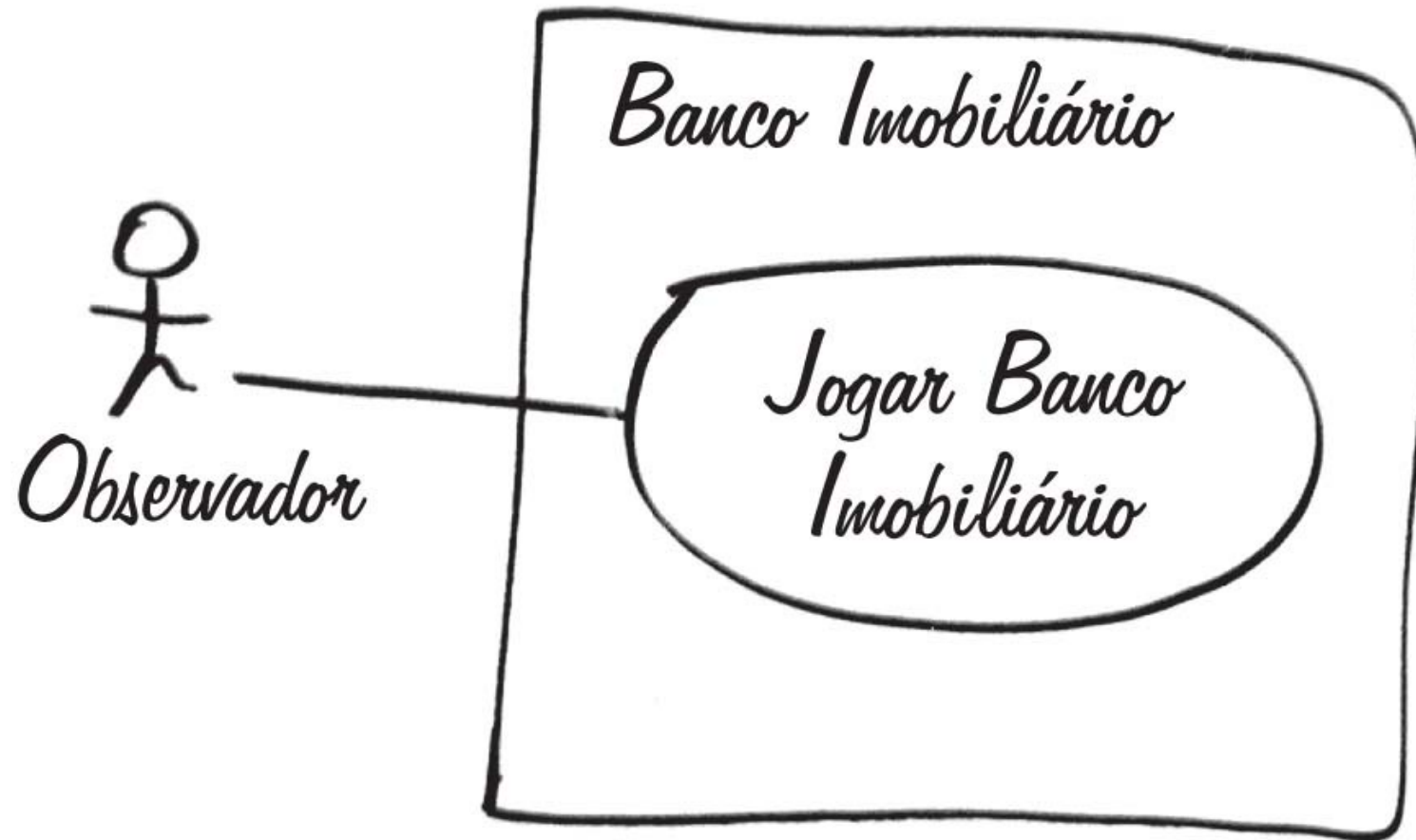


Diagrama de caso de uso



Caso de uso “Jogar banco imobiliário”

- **Principal Cenário de Sucesso:**

Extensões:

*a. Em qualquer momento o Sistema falha:

(para apoiar a restauração, Sistema registra depois de cada movimento completado)

1. Observador reinicia o Sistema.

2. Sistema detecta falha anterior, reconstrói estado e fica pronto para continuar.

3. Observador escolhe continuar (a partir da última vez completada pelo jogador).

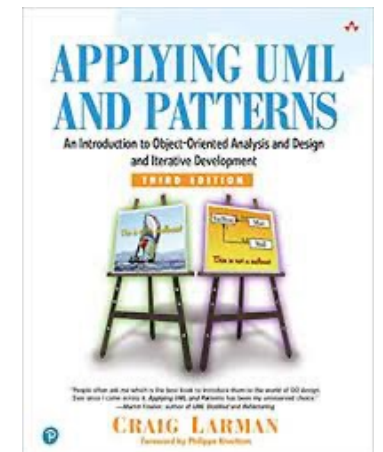
Caso de uso “Jogar banco imobiliário”

- Extensões:

Extensões:

- *a. Em qualquer momento o Sistema falha:
(para apoiar a restauração, Sistema registra depois de cada movimento completado)
 1. Observador reinicia o Sistema.
 2. Sistema detecta falha anterior, reconstrói estado e fica pronto para continuar.
 3. Observador escolhe continuar (a partir da última vez completada pelo jogador).
-

Capítulo 9: Modelos de Domínio



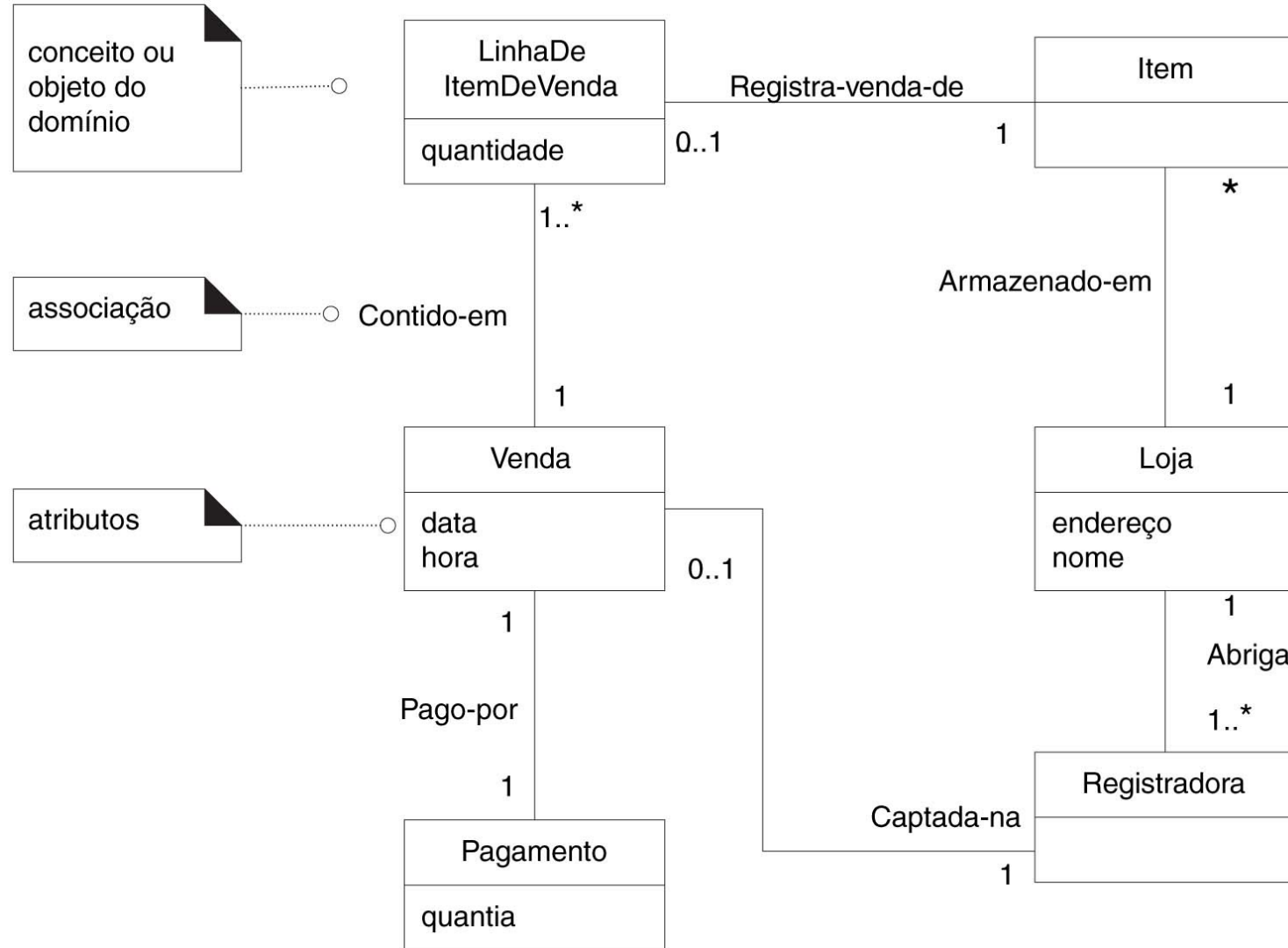
Objetivos

- Identificar **classes conceituais** relacionadas com os **requisitos** da **iteração corrente**.
- Criar um **modelo de domínio** inicial.
- Modelar os **atributos** e **associações** adequadas.

Introdução

- **Modelo de Domínio** é o modelo mais importante em **análise OO**:
 - Ilustra conceitos importantes de um **domínio**.
 - Inspiração para **projetar objetos** de software.
 - Utilizado como entrada para vários **artefatos**.
- O **modelo de domínio** é opcional.

Exemplo – Modelo de Domínio parcial

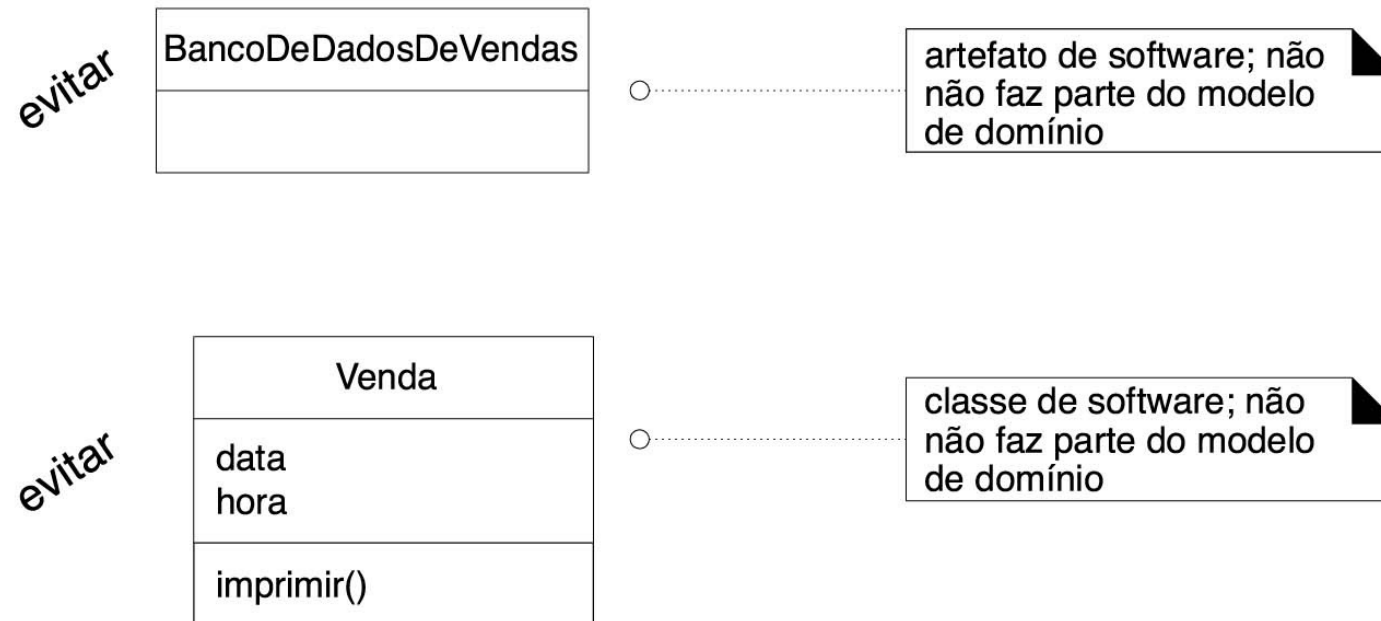


O que é um modelo de domínio?

“Um **modelo de domínio** é uma representação visual de **classes conceituais**, ou **objetos** do mundo real, em um **domínio**”. [Fowler96].

Um modelo de domínio é uma imagem de objetos de software?

- Um modelo de domínio não mostra artefatos ou classes de software



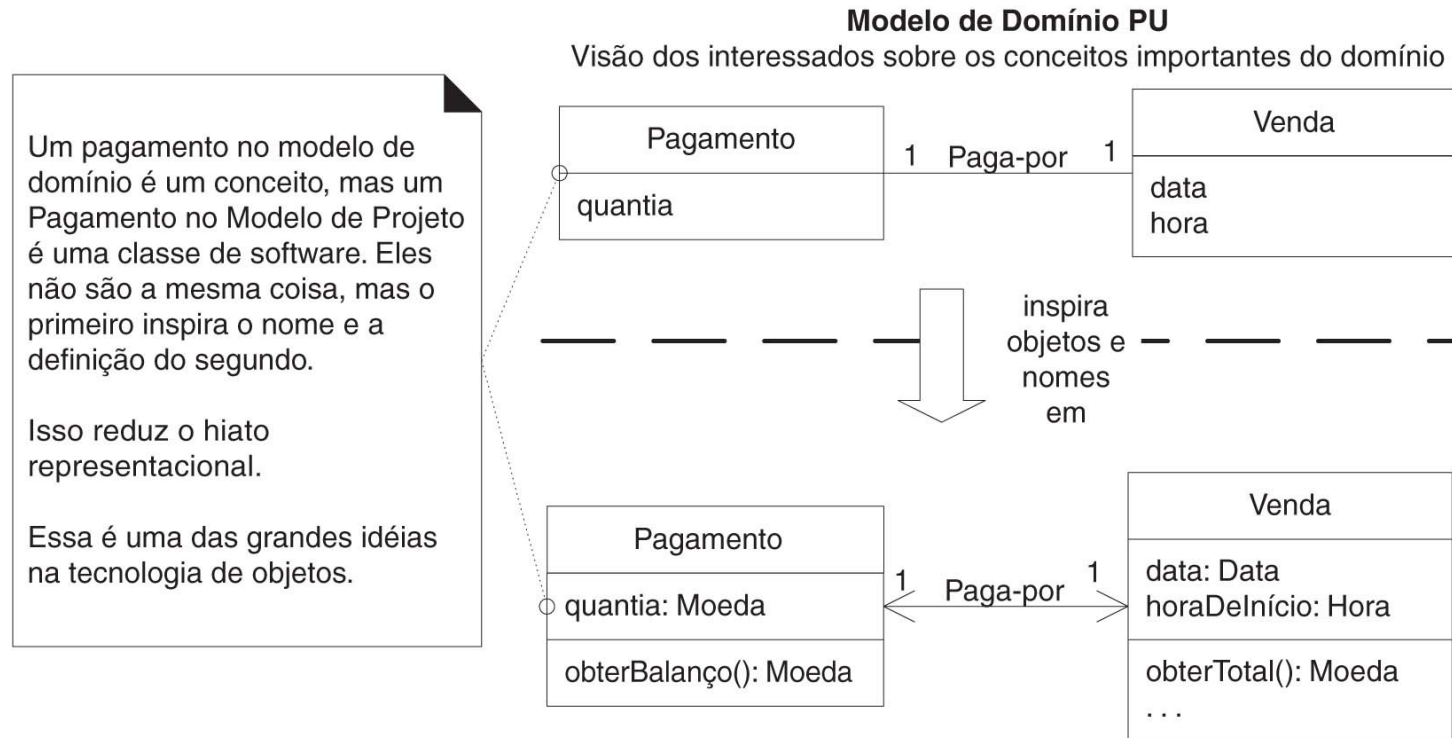
Dois significados tradicionais de “Modelo de Domínio?”

1. Uma perspectiva **conceitual de objetos** em uma situação real do mundo, não uma **perspectiva de software**
2. A **camada de domínio dos objetos de software**, isto é, a camada abaixo da **camada de apresentação**, composta de **objetos de domínio**
– **Exemplo:** classe de software *Tabuleiro* com um **método** *obterCasa*.

Dois significados tradicionais de “Modelo de Domínio?”

- **Ambas as definições estão corretas!**
- **Camada de domínio** será utilizada para indicar o **segundo significado** de **modelo de domínio**, que é o orientado a software

Menor hiato representacional com modelagem OO



Modelo de Projeto PU

O desenvolvedor orientado a objeto inspirou-se com o domínio do mundo real ao criar classes de software.

Portanto, o hiato representacional entre como os interessados concebem o domínio e sua representação no software foi reduzida.

Como criar um modelo de domínio?

- *Limitado* pelos **requisitos** na **iteração corrente**:
 1. Encontre as **classes conceituais**
 2. Faça seu **desenho** como **classes** em um **diagrama de classes UML**
 3. Acrescente **associações** e **atributos**

Como encontrar classes conceituais?

- Reutilizar ou modificar os **modelos existentes**.
- Identificar **substantivos** ou **frases nominais**.

Reutilizar ou modificar modelos Existentes

- É a melhor estratégia e geralmente a **mais fácil**
- Existem **modelos de domínio** e **modelos de dados** já publicados que foram bem concebidos para vários **domínios comuns**:
 - *controle de estoque*
 - *financeiro*
 - *de saúde*
 - *etc.*

Identificar substantivos ou frases Nominais

- **Análise linguística:**
 - identificar os **substantivos** e **frases nominais** nas **descrições textuais** em um **domínio**.
 - Considerá-los candidatos a **classes conceituais** ou **atributos**.
- Técnica útil por sua **simplicidade**.

Identificar substantivos ou frases nominais

- Casos de uso no formato completo são fontes para esta análise linguística.
- O cenário do caso de uso *Processar Venda* pode ser utilizado:

Cenário de Sucesso Principal (ou Fluxo Básico):

1. **Cliente** chega ao ponto de **pagamento do PDV** com as **mercadorias** e/ou **serviços** para adquirir.
2. **Caixa** inicia uma nova **venda**.
3. **Caixa** digita **identificador do item**.
4. Sistema registra **linha de item de venda** e apresenta uma **descrição do item**, seu **preço** e **total** parcial da venda. Preço é calculado de acordo com um conjunto de regras de preços.

Identificar substantivos ou frases Nominais

Caixa repete os passos 3 e 4 até que identifique ter terminado.

5. Sistema apresenta o total com os **impostos** calculados.

6. **Caixa** diz ao Cliente o total e solicita o **pagamento**.

7. Cliente para e Sistema trata o **pagamento**.

8. Sistema registra a **venda** completada e envia as informações de venda e pagamento para Sistema externo de **Contabilidade** (para contabilização e **comissões**) e Sistema de **Estoque** (para atualizar estoque).

9. Sistema apresenta **recibo**.

10. Cliente sai com recibo e mercadorias (se for o caso)

Extensões (ou Fluxos Alternativos):

...

7a. Pagamento com dinheiro:

1. Caixa digita a quantia de **dinheiro fornecida**.

2. Sistema apresenta o **valor do troco** e libera a **gaveta de dinheiro**.

3. Caixa deposita dinheiro fornecido e entrega o troco para Cliente.

4. Sistema registra o pagamento em dinheiro.

Identificar substantivos ou frases nominais

- O **modelo de domínio** é uma visualização de **conceitos** e **vocabulários** do **domínio** dignos de nota
- **Onde eles se encontram?**
 - *Nos casos de uso*
 - *Em outros documentos*
 - *Na cabeça dos especialistas*
- **Casos de uso** são uma **fonte rica** para identificar **frases nominais**

Identificar substantivos ou frases nominais

- O **ponto fraco** da abordagem é a **imprecisão da linguagem natural**:
 - Diferentes **frases nominais** podem representar a mesma **classe conceitual** ou **atributo**

Estudo de Caso: domínio PDV

Encontrar e desenhar as classes conceituais

- Lista restrita aos **requisitos** da **iteração 1**:
 - **Cenários** onde são consideradas apenas **vendas em dinheiro** de *Processar Vendas*.

<i>Venda</i>	<i>Caixa</i>
<i>PagamentoEmDinheiro</i>	<i>Cliente</i>
<i>LinhaDeItemDeVenda</i>	<i>Loja</i>
<i>Item</i>	<i>DescriçãoDoProduto</i>
<i>Registradora</i>	<i>CatálogoDeProdutos</i>
<i>LivroDiário</i>	

Encontrar e desenhar as classes conceituais

- **Não existe lista “correta”:**
 - Tem-se uma **coleção arbitrária de abstrações e vocabulários do domínio** consideradas dignas de nota.
- Seguindo as **estratégias de identificação**, listas similares serão produzidas por **diferentes modeladores**

Encontrar e desenhar as classes conceituais

- Na prática, não é necessária a criação de uma **lista textual**:
 - Pode-se desenhar um **diagrama de classes UML** a medida que as **classes conceituais** são descobertas



Estudo de Caso: domínio Banco Imobiliário

Encontrar e desenhar as classes conceituais

- Lista restrita aos **requisitos** da **iteração 1** de *Jogar Banco Imobiliário*:

BancoImobiliário

Jogador

Peca

Dado

Tabuleiro

Casa

Manter o modelo em uma ferramenta?

- Esquecer **classes conceituais** significativas durante a **modelagem** de **domínio** é normal:
 - Podemos descobri-las durante o **projeto** ou **programação**.
- Objetivos para criação de um **modelo de domínio**:
 - Entender rapidamente o **domínio**.
 - Comunicar um **esboço** de aproximação de **conceitos-chave**.

Manter o modelo em uma ferramenta?

- Não há motivo para manter ou atualizar o **modelo**:
 - Perfeição não é o objetivo
 - **Modelos ágeis** são descartados rapidamente.

Manter o modelo em uma ferramenta?

- Não significa que seja errado manter o **modelo** em uma **ferramenta**:
 - O **rascunho** pode ser redesenhado em uma **ferramenta CASE UML**.
 - O **desenho** original pode ser feito diretamente com uma **ferramenta** e um projetor acoplado ao computador:
 - Facilidade de visualização para toda a **equipe**

Como modelar o mundo irreal?

- Alguns **sistemas** se destinam a **domínios** que tem pouca analogia com **domínios naturais**:
 - *Ex: Software para telecomunicações*
- Criar um **modelo de domínio** nesses **domínios** requer:
 - Alto grau de **abstração**
 - Afastamento do que consideramos em **projetos não OO** mais habituais
 - Conhecer **conceitos-chave** e **vocabulários-chave** do **domínio**

Como modelar o mundo irreal?

- **Candidatas a classes conceituais relacionada ao domínio de dispositivos de chaveamento em telecomunicações:**
 - *Mensagem*
 - *Conexão*
 - *Porta*
 - *Diálogo*
 - *Rota*
 - *Protocolo*

Um erro comum relativo a atributos vs. classes

- Representar algo como **atributo** quando ele deveria ser uma **classe conceitual**.
- Regra para **evitar este engano**:

Diretriz

Se não pensamos em alguma classe conceitual X como um número ou texto no mundo real, X provavelmente é uma classe conceitual, não um atributo.

Um erro comum relativo a atributos vs. classes

- *Loja* deve ser um **atributo** de *Venda* ou uma **classe conceitual** *Loja*?

Venda
loja

ou... ?

Venda

Loja
númeroDe Telefone

Um erro comum relativo a atributos vs. classes

- No **mundo real**, uma **loja** não é considerada um **número** ou um **texto**, o termo sugere:
 - uma **entidade legal**,
 - uma **organização**
 - e **algo que ocupa espaço**
- Portanto *Loja* deve ser uma **classe conceitual**!

Um erro comum relativo a atributos vs. classes

- No domínio de reservas de passagens aéreas, *Destino* deveria ser um **atributo** de *Voo* ou uma **classe conceitual** *Aeroporto*?

Vôo
destino

ou... ?

Vôo

Aeroporto
nome

Um erro comum relativo a atributos vs. classes

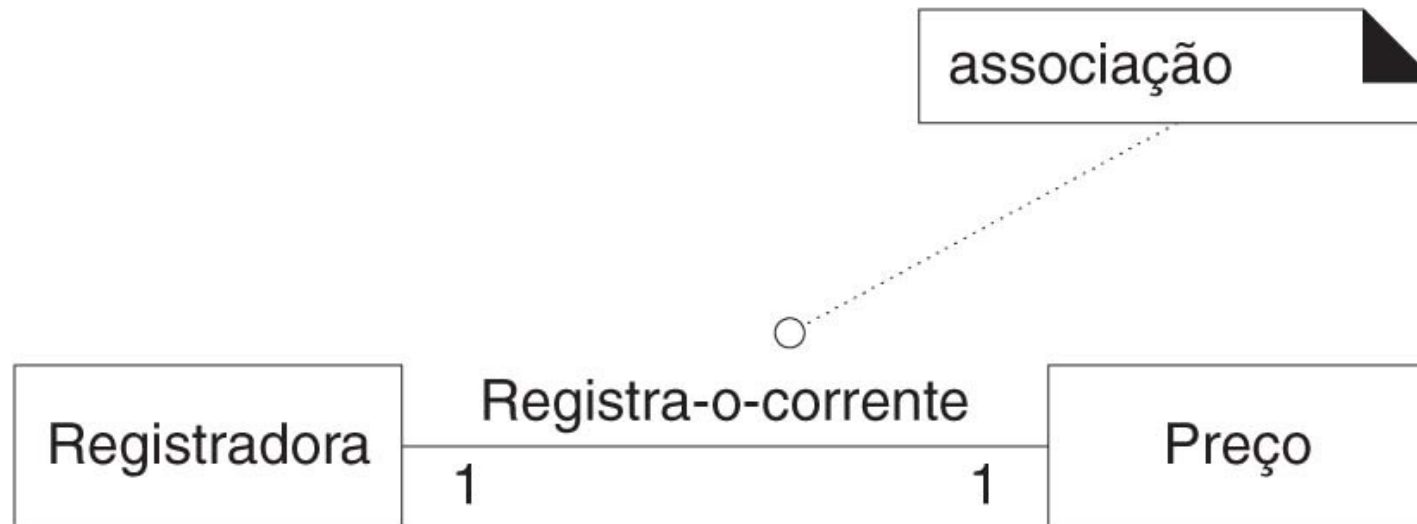
- No **mundo real**, um aeroporto de destino não é considerado um **número** ou um **texto**, ele é:
 - uma **coisa de grande porte**.
 - e **algo que ocupa espaço**.
- Portanto *Aeroporto* deve ser uma **classe conceitual**!

Associações

- É útil **encontrar e mostrar as associações**:
 - Satisfazer os **requisitos de informação** para os **cenários em desenvolvimento**.
 - Auxiliar na compreensão do **modelo de domínio**.

Associações

- Uma **associação** é um **relacionamento** entre **classes** (instâncias destas **classes**) que indica alguma **conexão significativa e de interesse**.



Quando Mostrar uma Associação?

- Quando...
 - Um **relacionamento** necessita ser **preservado por algum tempo**:
 - **Milissegundo ou anos** (depende do contexto).

Quando mostrar uma associação?

- **Ou...**
 - Necessitamos ter alguma **memória de um relacionamento** entre **objetos**:
 - **Exemplo:** Precisamos lembrar quais **instâncias** de *LinhaDeItemDeVenda* são **associadas** a uma **instância** de *Venda* para ser possível:
 - *Reconstruir a venda*
 - *Imprimir um recibo*
 - *Calcular o total da venda*
 - Precisamos lembrar as *Vendas* **completadas** em um *Livro Diário* para **fins legais e contabilidade**.

Quando mostrar uma associação?

- No **domínio** do jogo *Banco Imobiliário* é preciso lembrar:
 - Em qual *Casa* uma *Peça* (ou *Jogador*) está.
 - Quais *Casas* são **parte** de um *Tabuleiro* em particular.

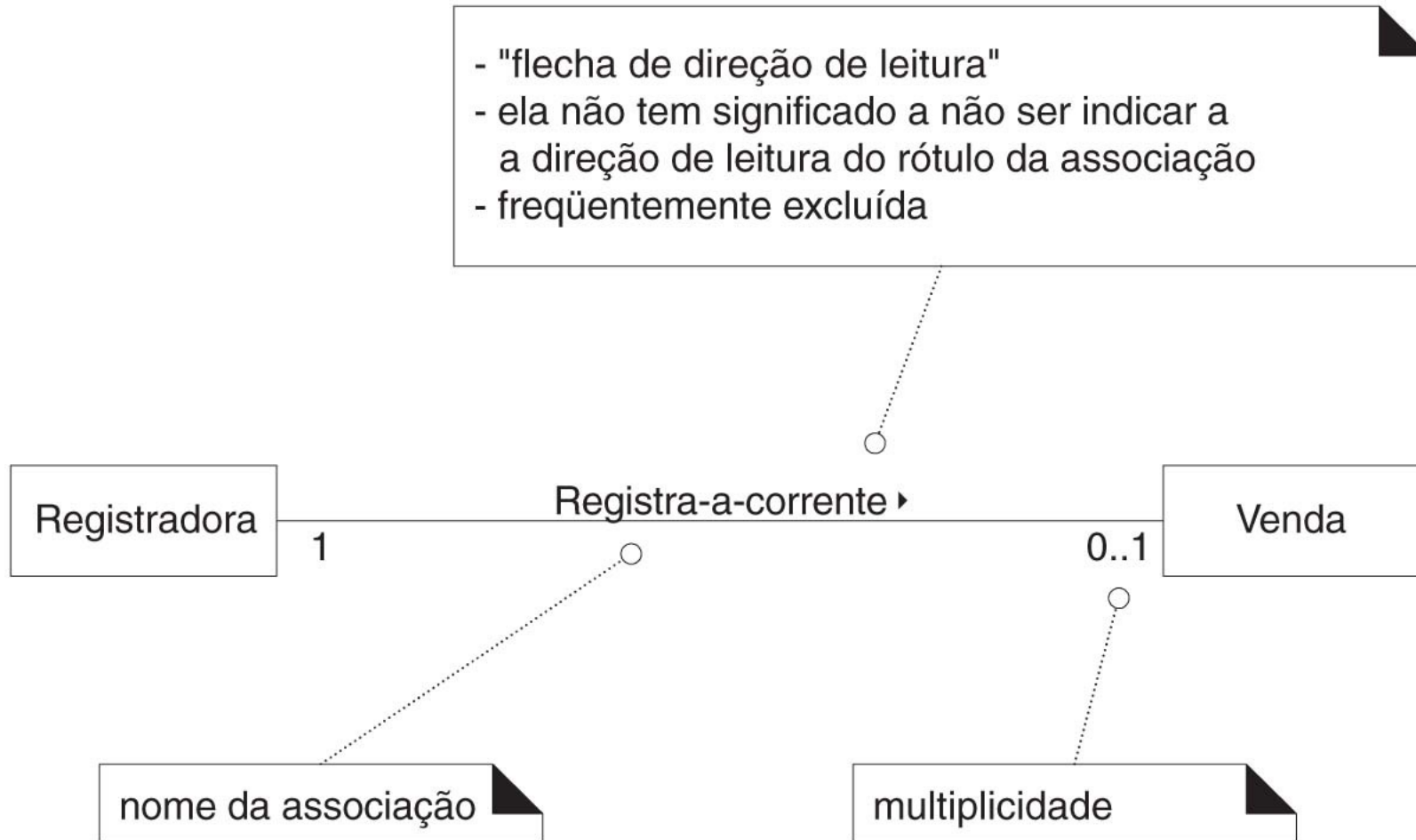
Aplicação da UML: notação para associação

- Uma **associação** é representada como uma **linha entre classes**
- O **nome da associação** deve ser escrito em **letra maiúscula**
- Os **extremos** de uma **associação** pode conter uma **expressão de multiplicidade**:
 - Indica o **relacionamento numérico** entre as **instâncias das classes**

Aplicação da UML: notação para associação

- A **associação** é inerentemente **bidirecional**:
 - O **percurso lógico** é possível a partir de uma **instância** de qualquer **classe**, em **ambos os lados**.
- Este **percurso** é puramente **abstrato**:
 - Não é uma **afirmação** sobre **conexões** entre **entidades de software**

A Notação UML para Associação

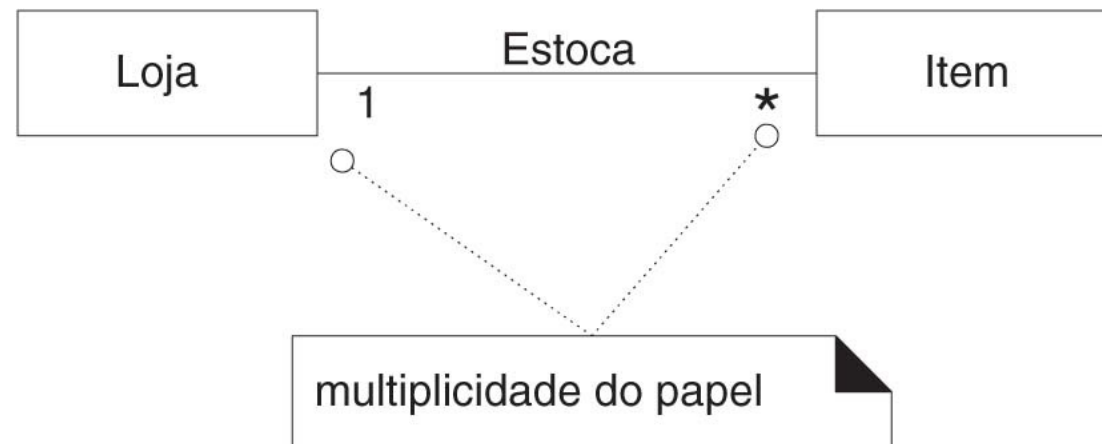


Aplicação de UML: papéis

- Cada **extremo de uma associação** é chamado de **papel**.
- Os **papéis** podem ter:
 - *Expressão de multiplicidade*
 - *Nome*
 - *Navegabilidade*

Aplicação de UML: multiplicidade

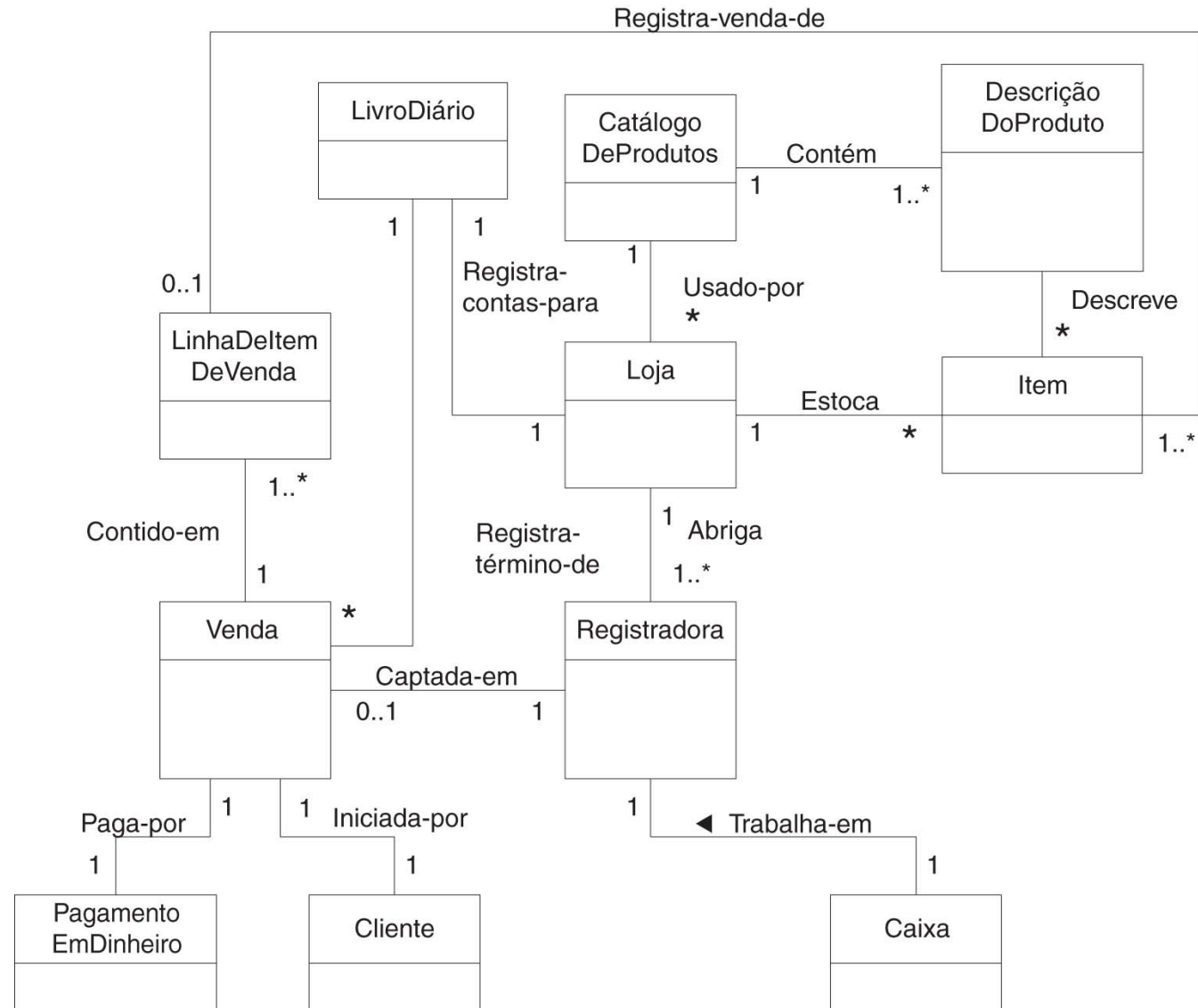
- A **multiplicidade** define quantas **instâncias** de uma **classe A** podem estar **associadas** a uma **instância** de uma **classe B**.
- Uma única **instância** de *Loja* pode ser **associada** a **zero ou mais** (indicado por *****) **instâncias** de *Item*



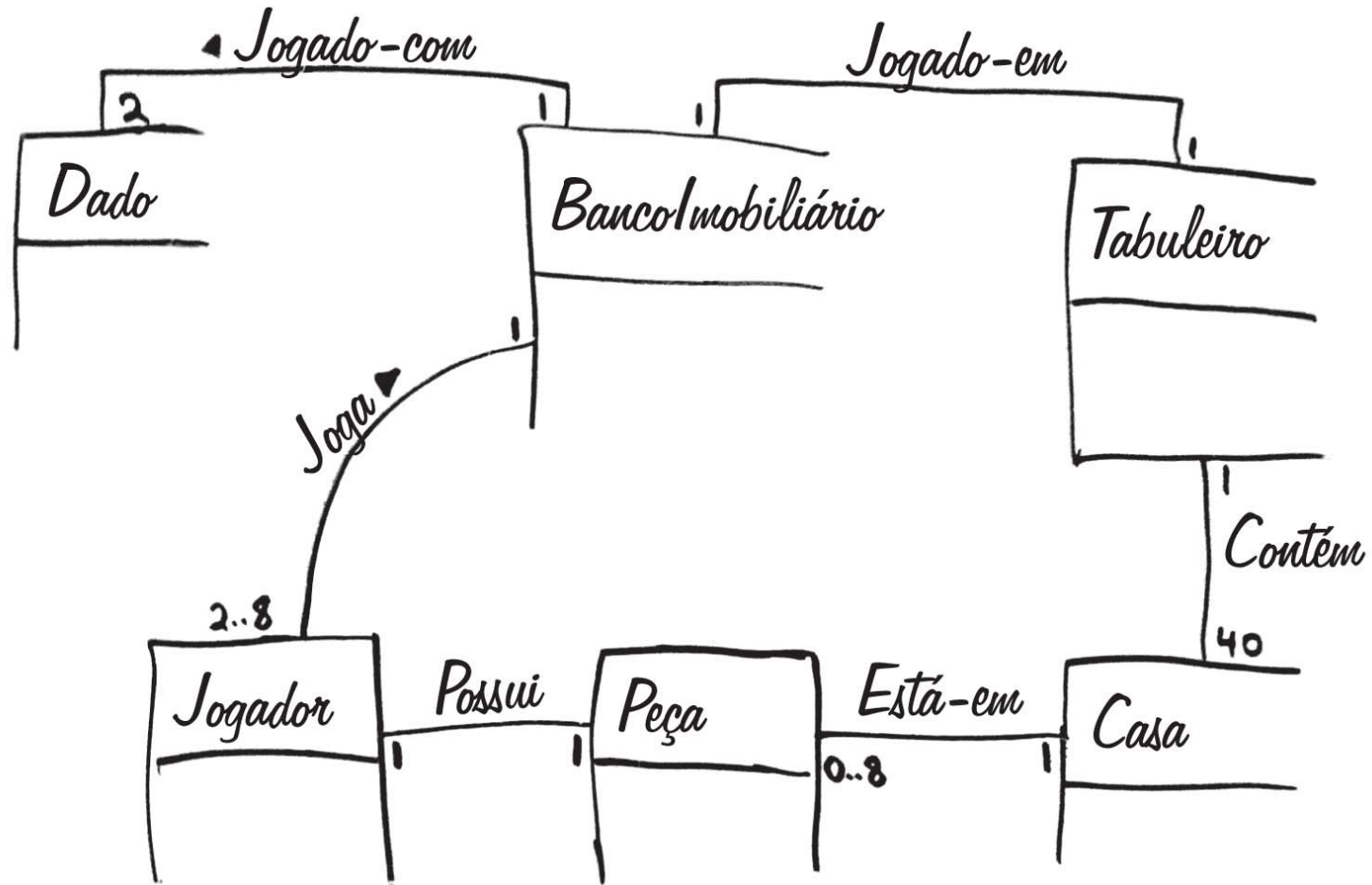
Exemplos de expressões de multiplicidade

*	T	zero ou mais; "muitos(as)"*
1..*	T	um ou mais
1..40	T	um a 40
5	T	exatamente 5
3, 5, 8	T	exatamente 3, 5 ou 8

Associações nos modelos de domínio



Associações nos modelos de domínio



Atributos

- Um **atributo** é um **valor de dados lógico** de um **objeto**.
- Identificar os **atributos** das **classes conceituais**:
 - Satisfazer os **requisitos** dos **cenários** em **desenvolvimento**.

Quando mostrar atributos?

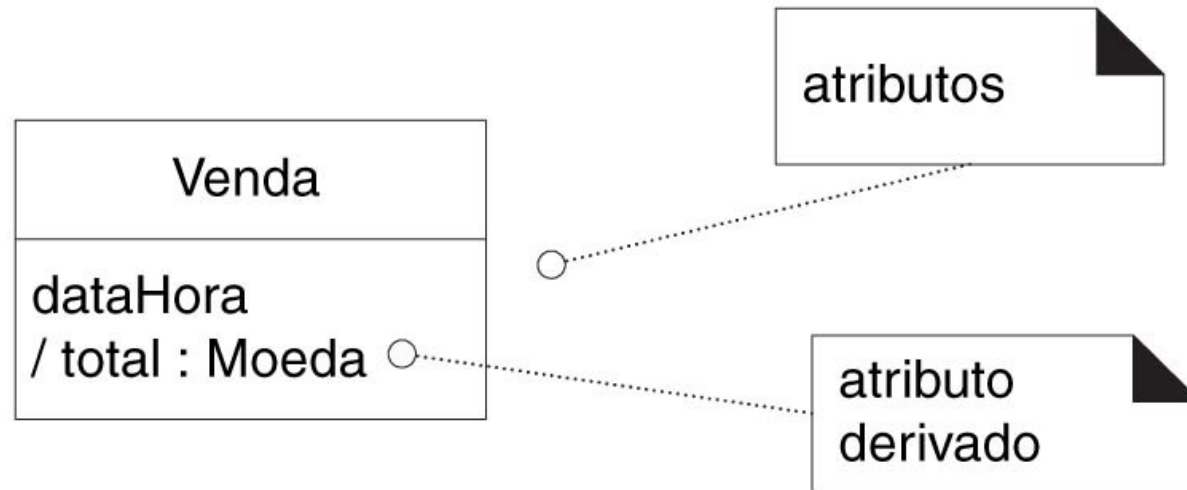
- **Quando...**
 - Sugerem ou implicam a necessidade de **memorizar informações**.

Quando mostrar atributos?

- **Exemplo:**
 - Um **recibo** (contém informações de *Venda*)
 - O **caso de uso** *ProcessarVenda* inclui uma **data** e uma **hora**, o **nome** e **endereço da loja** e a **ID do caixa**
 - **Consequentemente:**
 - *Venda* precisa de um atributo *dataHora*
 - *Loja* precisa de um *nome* e *endereço*
 - *Caixa* precisa de uma *ID*

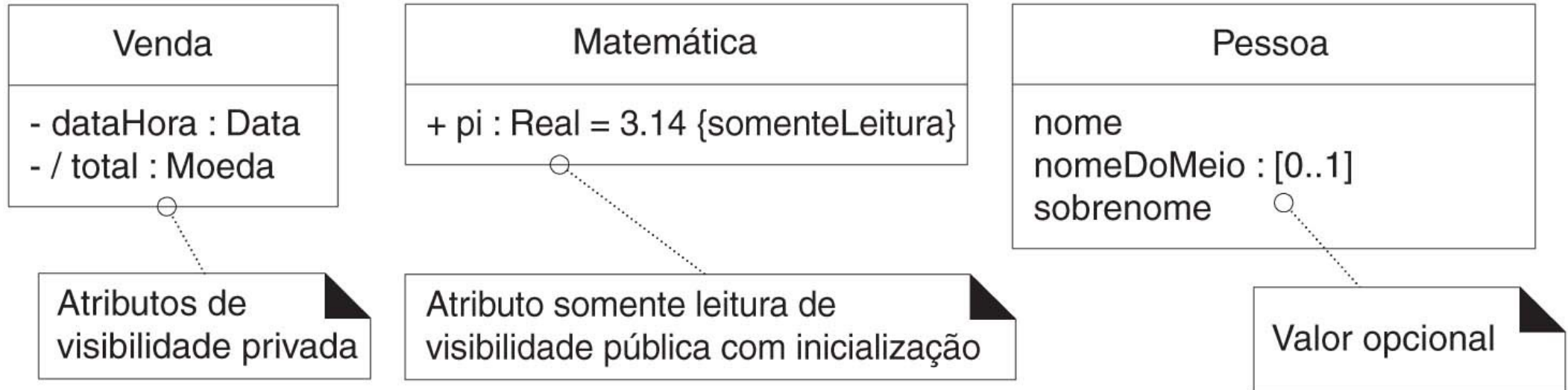
Notação de atributos

- Os **atributos** são mostrados no **segundo compartimento** da caixa da **classe**.
- Mostrar o seu **tipo** e **outras informações** é opcional



Mais notação

- Exemplos comuns:



Mais notação

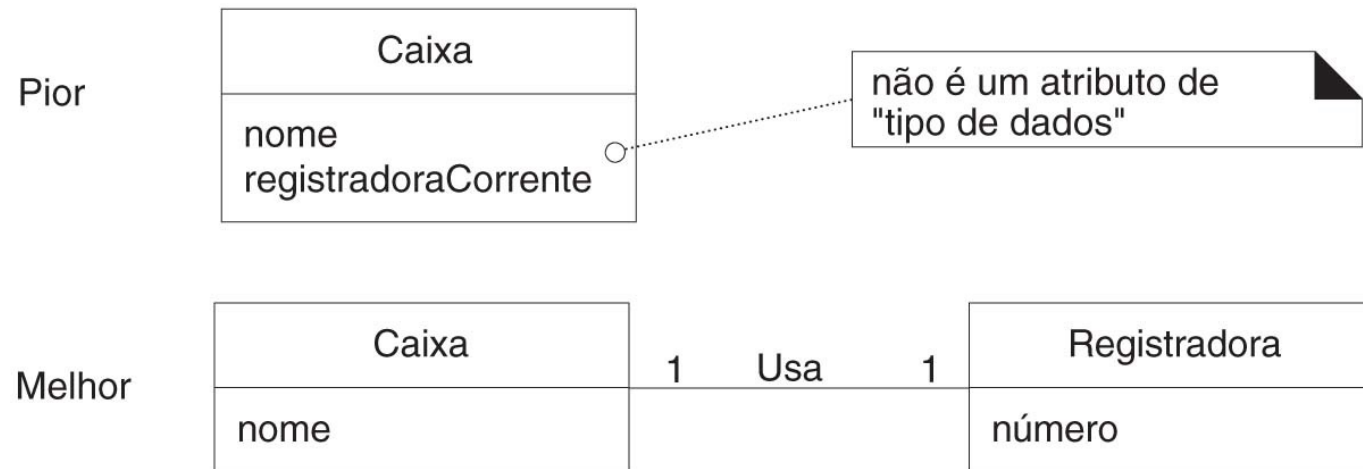
- A maioria dos modeladores considera os **atributos** com **visibilidade privada (-)**:
 - Não é necessário desenhar o **símbolo explícito** de **visibilidade**.

Quais os tipos de atributos adequados?

- A maioria dos **atributos** são os conhecidos como “**primitivos**”:
 - *Números, booleanos, etc.*
- O **tipo de um atributo** não deve ser um conceito complexo do **domínio**:
 - *Venda, Aeroporto.*

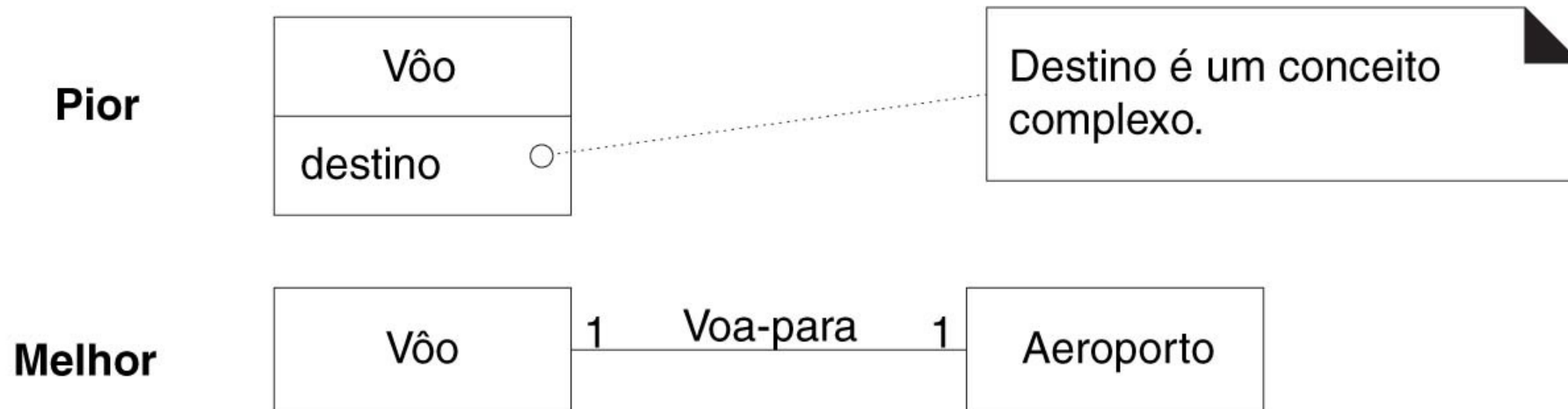
Quais os tipos de atributos adequados?

- O **atributo** *registradoraCorrente* na classe *Caixa* é indesejável:
 - Não é um **tipo simples de atributo**.
- **Maneira melhor:**
 - **Associar** *Caixa* e *Registradora*



Quais os tipos de atributos adequados?

- **Confusão comum:**
 - Modelar um **conceito complexo do domínio** com um **atributo**



Quais os tipos de atributos adequados?

Diretriz

Relacione classes conceituais com uma associação, não com um atributo.

Como funcionam os atributos no código?

- Recomenda-se que os **atributos** no **modelo de domínio** sejam **primitivos**:
 - Não significa que **atributos** em **C#** ou **Java** deve ser somente **dados simples**
 - O **modelo de domínio** é uma **perspectiva conceitual**, não de **software**
- No **Modelo de Projeto**, os **atributos** podem ser de **qualquer tipo**

Quando definir novas classes de tipos de dados?

- Os **atributos** *preço* e *quantia* devem ser uma **classe** do **tipo de dados** *Dinheiro*:
 - São **quantidades** em uma unidade monetária.
- O **atributo** *endereço* deve ser uma **classe** do **tipo de dados** *Endereço*:
 - Possui **seções separadas**

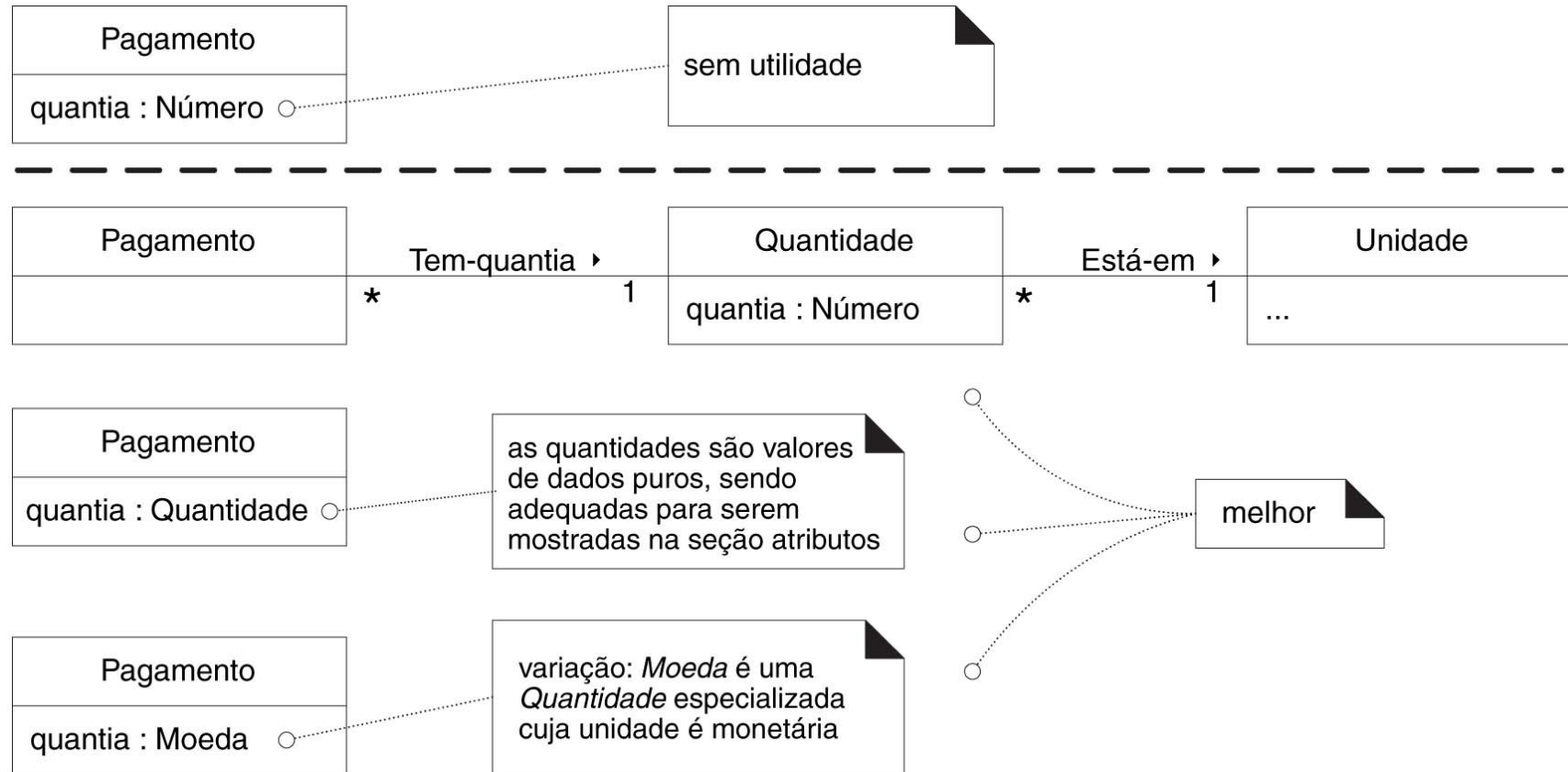
Modelagem de quantidades e unidades

- Muitas **quantidades numéricas** não podem ser **representadas** com **números simples**:
 - *Preço*
 - *Peso*
- Dizer que “**o preço era 13**” ou o “**peso era 27**” não diz muito:
 - *Reais? Quilogramas?*
- É necessário **conhecer as unidades** para que possam ser **convertidas**

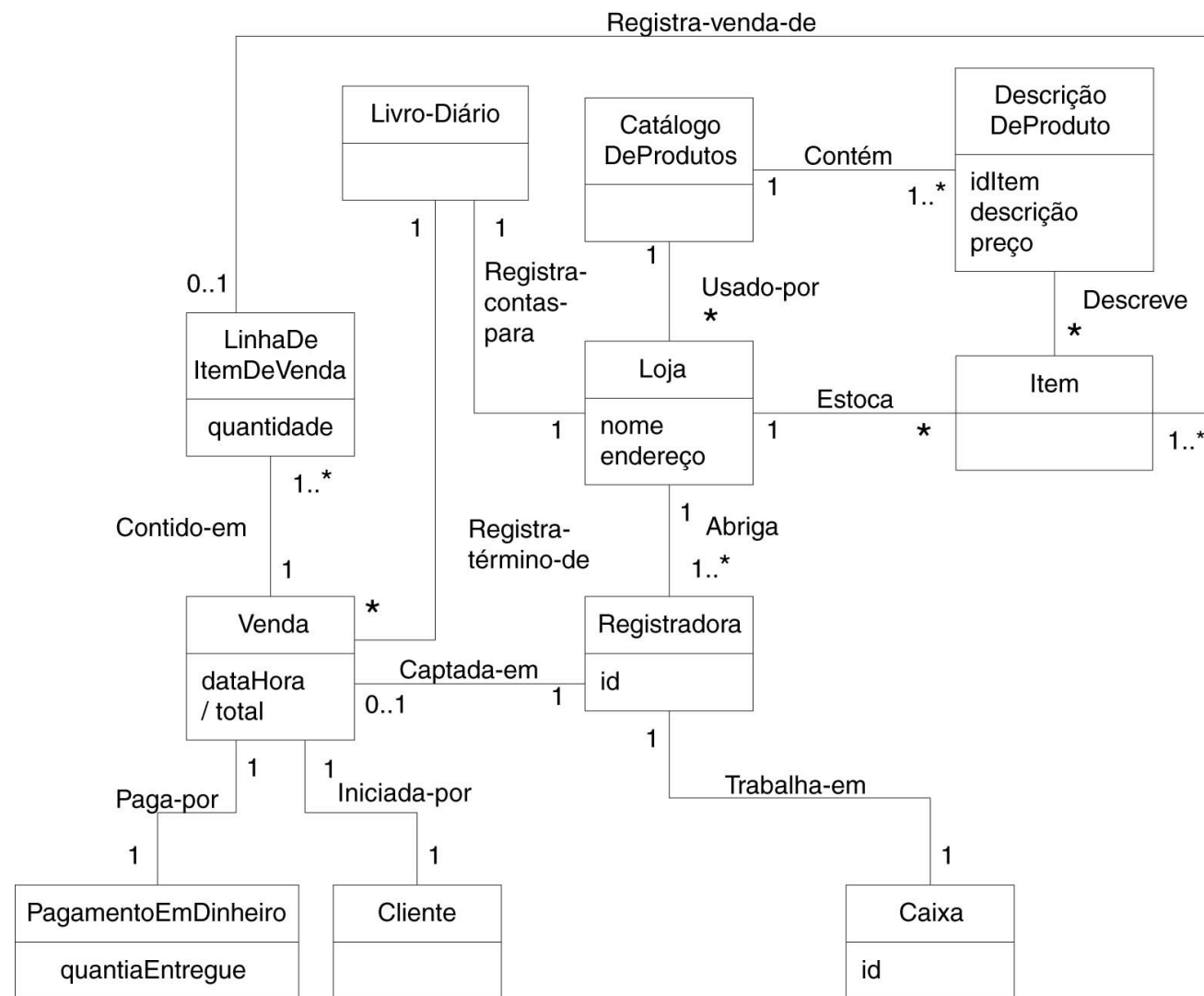
Modelagem de quantidades e unidades

- O **PDV** destina-se ao **mercado internacional** e deve aceitar **preços de diversas moedas**:
 - Na maioria dos casos, a solução é **representar** *Quantidade* como uma **classe distinta**, com uma *Unidade associada* [Fowler96]:
 - *Dinheiro* é um **tipo de quantidade** cuja **unidades** são moedas.
 - *Peso* é uma **quantidade** com **unidades** como quilo ou libras.

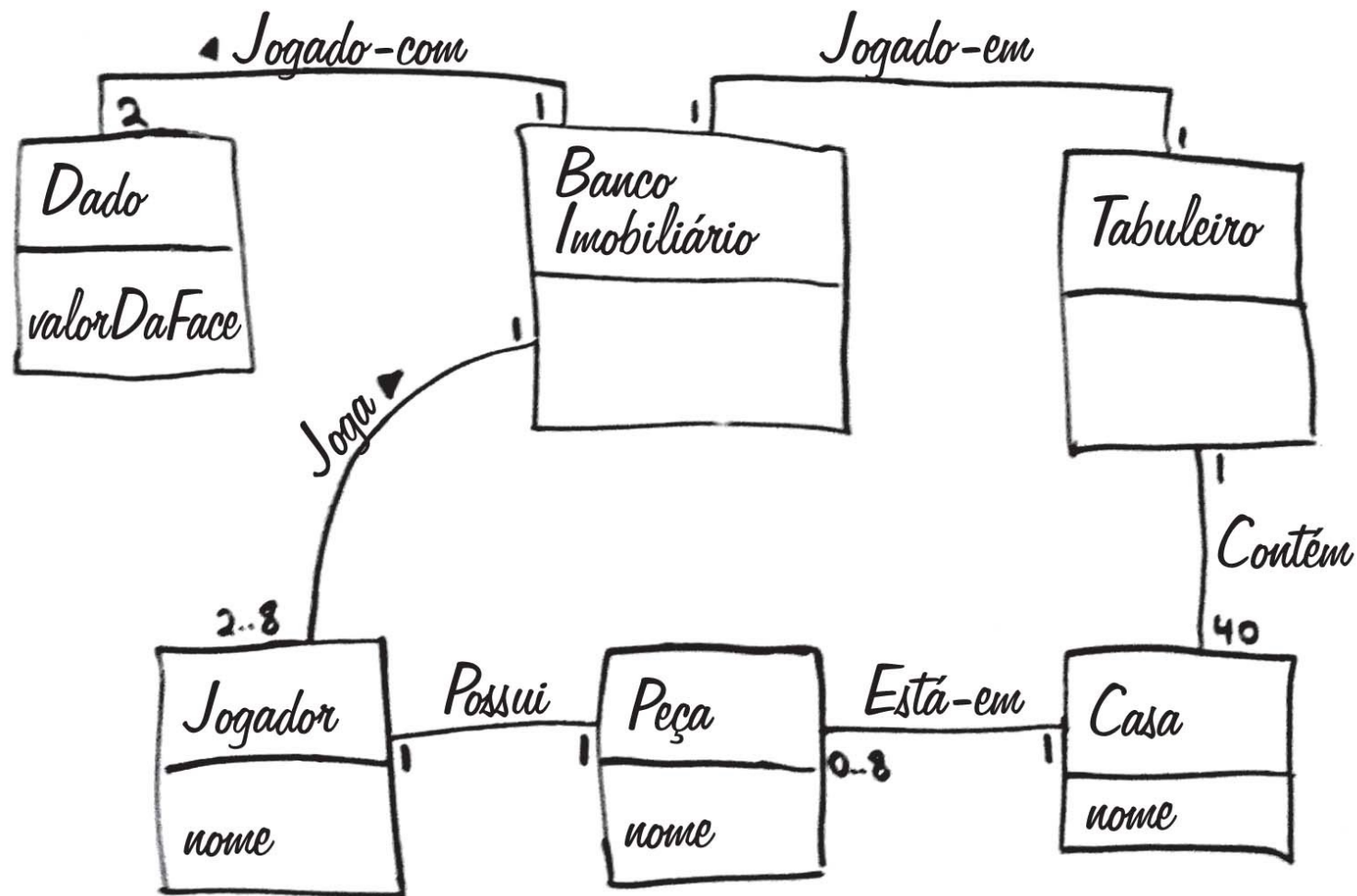
Modelagem de quantidades



Modelo de domínio parcial do PDV ProxGer



Modelo de domínio parcial do Banco Imobiliário



Dúvidas?

